VoxTool: LLM-Assisted Annotation — Project Plan

STATUS: PROPOSED · SEPTEMBER 2026 · VOXTOOL · PENN NEUROBRIDGE

1. Goal

Add an LLM to VoxTool whose single job is to **read the implant documentation PDF and produce the lead definitions**, so the annotator no longer transcribes them by hand. The human continues to pick every contact.

Concretely, the LLM replaces this manual step:

> Open the PDF → read off each lead (name, type, number of contacts) → type all of it into the *Define leads* panel → double-check nothing was mistyped.

with:

> Drop in the PDF → review a pre-filled lead table beside the source text → confirm or correct → start annotating.

The measurable win is **fewer transcription errors**, not a large time saving. A mistyped lead name or wrong contact count currently flows straight into the saved `voxel_coordinates.json` with nothing to catch it. Secondary win: the different documentation formats produced by different recording systems get normalised into one schema.

2. Scope decision: what the LLM does and does not do

This was the central decision of the planning meeting and it is deliberate.

Boundary	What it covers
In scope	Extracting structured lead definitions from unstructured documents. Normalising vendor-specific formats into VoxTool's schema.
Out of scope	Deciding which bright voxel clusters are electrode contacts. Placing or adjusting coordinates. Anything that writes a coordinate without a human clicking it.

Rationale. Distinguishing contacts from surgical wires, staples and other metal in a CT is a 3D computer-vision problem, not a language problem. Deep leads sit close to other hardware and a language model given that job will be confidently wrong some fraction of the time. In a surgical planning tool, a plausible-looking wrong coordinate is worse than no suggestion at all. Keeping the human as the one who points at contacts is both more robust and far easier to defend in review.

The LLM is therefore a **typist, not an authority**. Its output is always shown next to the source and always requires confirmation.

3. What already exists (so effort is scoped honestly)

VoxTool today is a React frontend (`web/frontend`) with a Flask backend (`web/backend`), deployed two ways: a cloud instance on CloudFront and an Electron desktop app (currently `v1.0.7`) that bundles the same frontend with a frozen backend.

Already working, and **not** part of this project:

- **Semi-automated contact tracing.** `Interpolate` already does "human marks the first and last contact, the tool fills in everything between and snaps each one to nearby bright voxels." The workflow described in the meeting as the target is largely already shipped.
- Contact marking with intensity-based snapping (`/snap`), coordinate conversion (`/mm_to_voxel`, `/voxel_to_mm`), 2D slice and 3D electrode views, save/load as JSON and legacy TXT.
- A lead schema (`name`, `type`, `dimensions`) — **this is already the LLM's output target**, so the format does not need designing.

This is why the extraction work is a small addition rather than a rewrite. The new surface area is one backend endpoint, one review screen, and a decision about where the model runs.

4. Questions that gate the work

These need answering before implementation, because the answers change the architecture. They are not engineering tasks.

1. **Are the PDFs digital text or scanned images?** Digital text is straightforward. Scans require OCR, which introduces a new dependency and a new failure mode (`LA1` misread as `LAI` or `LA l` silently corrupts a lead name). This single question roughly doubles or halves the scope.
2. **What is the compliance path for patient documents?** The implant PDFs contain identifiers. Sending them to a third-party API requires either a BAA covering that provider or de-identification before the document leaves the machine. Note the cloud deployment currently has no authentication and is documented as demo-only. **If there is no BAA route, the local model is not an optimisation — it is the only option**, and cloud extraction has to be restricted to de-identified documents.
3. **How many distinct document formats are in play, and can we get examples of each?** Nothing below can be built or evaluated without real samples.
4. **Who are the users?** Penn only, or distributed to other centres? This decides whether local-first or cloud-first is the priority.

5. Phased plan

Effort estimates assume part-time student capacity, not full-time work. Each phase has an exit criterion so progress is visible without guessing.

Phase 0 — Samples and ground truth *(no code)*

- Collect 5–10 de-identified example PDFs spanning every recording system in use.
- Hand-write the correct lead definitions for each one. This becomes the **evaluation set** used for every later decision.
- Record for each sample: digital text or scan, page count, whether the lead table is a real table or prose.

Exit: an eval set exists, checked into the repo (or a secure share if the documents cannot be committed), with expected output for each sample.

This is the real blocker. Everything after it is comparatively mechanical.

Phase 1 — Extraction schema and backend endpoint

- Define the extraction JSON schema formally. Starting point is the existing lead schema, plus a per-field confidence or "not found" marker so gaps are explicit rather than guessed.
- Add `POST /api/extract/leads` to the Flask backend: PDF in, schema-valid JSON out. Text extraction via PyMuPDF; OCR only if Phase 0 shows it is needed.
- Implement it behind a **provider interface** with three backends:
  1. `cloud` — hosted API, best accuracy
  2. `local` — open-weight model on the user's machine
  3. `manual` — no extraction, the existing hand-entry path

  The application must never *depend* on a model being available. Extraction pre-fills a form; manual entry stays supported permanently.
- Add deterministic validation independent of the model: contact counts are positive integers, `dimensions` multiply out to the contact count, lead names match an expected pattern. Where the rules and the model disagree, surface it for the human rather than silently choosing.

Exit: the endpoint returns valid JSON for every Phase 0 sample using the cloud backend, and accuracy against ground truth is measured and written down.

Phase 2 — Review and confirm UI

- New step in the frontend: upload PDF → extracted lead table shown **beside the source text it came from** → edit any row → confirm.
- Confirming populates the existing *Define leads* state. Nothing is written without an explicit confirm.
- Every extracted field is editable. Anything the model could not find is shown blank and flagged, never filled with a guess.

Exit: a full run on a real scan — PDF in, leads confirmed, contacts marked by hand, coordinates saved — with no manual lead typing.

Phase 3 — End-to-end pipeline closure

Per the meeting's preference for closing the loop before adding features:

- Save destinations: local disk (already works) **and** a configured Penn location.
- Implement the Penn upload as a **configuration setting plus a button**, not a natural-language instruction. Deliberate deviation from the meeting — see §7.
- Confirm the whole path works identically in the desktop app and the cloud build.

Exit: one documented end-to-end run on each of cloud and desktop.

Phase 4 — Local model evaluation and packaging

Only now, with an eval set and a working pipeline, does model choice get decided by measurement rather than assumption.

- Benchmark small open-weight models (Llama, Qwen, Mistral class) against the Phase 0 eval set. The meeting's assumption that "it is a simple task so a small model will do" is plausible but **unverified** — multi-vendor clinical documents are not necessarily clean.
- Use **grammar-constrained decoding** (llama.cpp GBNF or equivalent) so output is guaranteed schema-valid. This eliminates malformed output as a failure class, leaving only wrong values, which validation and human review catch.
- Packaging constraint discovered up front: a 7–8B model at 4-bit is ~4–5 GB, against current installers of 104–129 MB. **GitHub release assets are capped at 2 GB per file**, so a bundled model cannot ship through the existing `build-desktop.yml` release pipeline. The model must be a first-run download, or the app depends on a local runtime (e.g. Ollama) being installed.
- Runtime expectation: 7B at 4-bit on CPU is roughly 5–15 tokens/sec, so a few hundred tokens of JSON takes 30–100 seconds on the older laptops actually in use. Acceptable for a once-per-patient step **only if** it runs as a background job with progress, matching how the threshold cloud build already behaves.

Exit: a measured accuracy comparison of local vs cloud on the same eval set, and a packaging decision justified by those numbers.

6. Cloud vs local

Both are achievable, and the provider interface in Phase 1 means the same schema and the same review UI serve both. The differences:

Consideration	Cloud API	Local open-weight
Accuracy	Highest	To be measured (Phase 4)
Cost	Cents per patient — effectively negligible	Zero marginal, one-off engineering
Patient data	Leaves the machine; needs BAA or de-identification	Never leaves the machine
Setup for user	Needs an API account/key	None once installed
Speed	Seconds	30–100 s on older hardware

Cost should not drive this decision. A PDF is roughly 5–20k input tokens and under 1k output; even a thousand patients is a rounding error. Compliance and distribution are the deciding factors, not money.

7. Deliberate deviation from the meeting

The meeting suggested replacing GUI save buttons with prompting the LLM ("save it to Penn").
Recommend against.

Saving is a repeated, high-stakes, well-defined action. Routing it through a language model makes it less predictable and adds a failure mode where a patient file goes somewhere nobody intended, in exchange for no capability that a dropdown and a button do not already provide. The principle worth holding: **use the LLM only where the input is genuinely unstructured** — which is the PDF, and nothing else.

Happy to revisit if the goal is a conversational interface as a research contribution in itself rather than a usability improvement.

8. Explicitly out of scope for v1

Recorded so the boundary is not relitigated mid-build:

- Automatic identification of electrodes or contacts from the CT.
- Any agentic system that annotates without human pointing.
- A classifier distinguishing electrode clusters from other metal.
- Natural-language control of file operations.

These are the natural v2 direction and the meeting's own view was that we are not there yet. The v1 pipeline is a prerequisite for attempting them anyway, since it produces the confirmed lead definitions any automated approach would need as input.

9. Risks

Risk	Impact	Mitigation
PDFs turn out to be scans	Scope grows substantially	Resolve in Phase 0 before committing
No BAA route for patient documents	Cloud extraction unusable	Provider interface means local backend can carry the feature alone
Small local model insufficiently accurate	Packaging plan invalid	Eval set exists before model choice; cloud remains a fallback
Model output silently wrong	Corrupted coordinates	Mandatory human confirm, side-by-side source, deterministic cross-checks
Model download / size	Cannot use existing release pipeline	First-run download or external runtime, decided in Phase 4

10. Relevance to the paper

The end-to-end pipeline is what makes this describable: an LLM-assisted annotation tool where document understanding is automated, contact selection stays human, and the same workflow runs offline on a clinician's laptop or in the browser. The scoping decision in §2 — using the model only where it is reliable, and refusing it where it is not — is a defensible contribution in itself, and the Phase 0 eval set gives concrete numbers to report rather than claims.